IRIS FLOWER CLASSIFICATION USING MACHINE LEARNING

Data Science and Machine Learning Internship Capstone Project

Prepared by	Sayantani Adhikary
Project type	Data Science and Machine Learning
Programming language	Python
Development platform	Google Colab
Code repository	github.com/adhikarysayantani7-tony/aicte-internship-work

Final Project Report

This report documents the dataset, workflow, model implementation, evaluation and findings of the Iris flower classification project.

Abstract

This project applies supervised Machine Learning to classify Iris flowers into three species: Iris Setosa, Iris Versicolor and Iris Virginica. The dataset contains 150 records and four numerical measurements: sepal length, sepal width, petal length and petal width. Two classification algorithms, Decision Tree and K-Nearest Neighbors (KNN), were trained and evaluated using a held-out test set. In the experiment, Decision Tree achieved 93.33% accuracy and KNN achieved 100.00% accuracy. KNN performed better on this particular test split.

1. Project Overview

The Iris flower classification problem is a well-known introductory Machine Learning task. The aim is to use measurements of a flower to predict its species. This project follows a basic Machine Learning workflow: loading data, exploring and visualizing it, separating features and labels, splitting the data, training models and evaluating their predictions.

2. Project Objectives

The objectives of this project are to:

• Understand and explore the Iris dataset.

• Perform Exploratory Data Analysis (EDA) and visualize selected patterns.

• Prepare the input features and target labels for Machine Learning.

• Train two classification models: Decision Tree and K-Nearest Neighbors.

• Evaluate and compare the models using accuracy and classification metrics.

3. Dataset Description

The project uses the Iris dataset from the UCI Machine Learning Repository. It contains 150 flower records, with 50 records for each of the three species.

Feature	Description
Sepal length	Length of the sepal
Sepal width	Width of the sepal
Petal length	Length of the petal
Petal width	Width of the petal
Target	Species: Setosa, Versicolor or Virginica

Dataset source: UCI Machine Learning Repository — https://archive.ics.uci.edu/dataset/53/iris

4. Tools and Technologies

• Python — programming language.

• Google Colab — cloud-based notebook environment.

• Pandas and NumPy — data loading and numerical/data handling.

• Matplotlib — data visualization.

• Scikit-learn — data splitting, model training and evaluation.

• GitHub — project version control and repository hosting.

5. Methodology and Project Workflow

5.1 Dataset loading

The CSV dataset was loaded into a Pandas DataFrame. The data was checked by displaying sample rows, column names, dataset dimensions and summary information.

5.2 Data exploration and preparation

The dataset was examined using descriptive statistics and species counts. Missing-value checks were performed. The dataset contains 150 records, five columns including the target species, and no missing values were found in the checks performed.

5.3 Exploratory Data Analysis

A species-count bar chart and a scatter plot comparing petal length with petal width were created to help understand the distribution of the species and the relationship between two flower measurements.

5.4 Features, target and train-test split

The four numerical measurements were used as input features (X), and the species column was used as the target (y). The data was divided into training and testing sets using an 80:20 split, random_state=42 and stratify=y. This produced 120 training records and 30 testing records.

5.5 Model training

A DecisionTreeClassifier was trained with random_state=42. A KNeighborsClassifier was also trained using n_neighbors=5. Both models used the same training and testing split for comparison.

6. Model Evaluation and Results

Accuracy measures the proportion of test records predicted correctly. The observed test-set results were:

Model	Test accuracy
Decision Tree Classifier	93.33% (28 out of 30)
K-Nearest Neighbors (KNN)	100.00% (30 out of 30)

The KNN confusion matrix showed 10 correct predictions for each species in the 30-record test set: 10 Setosa, 10 Versicolor and 10 Virginica. The KNN classification report showed precision, recall and F1-score of 1.00 for each class on this test set.

The Decision Tree accuracy was 93.33%, meaning 28 of the 30 test records were classified correctly. KNN achieved a higher accuracy on this split. These results describe this particular test set and do not guarantee the same accuracy on all unseen data.

7. Comparison and Discussion

Both algorithms performed well on the Iris test set. KNN achieved 100.00% accuracy, while Decision Tree achieved 93.33%. Therefore, KNN was the better-performing model in this experiment. The comparison graph in the notebook visually presents these two accuracy values. A single train-test split is a limited evaluation, so further testing such as cross-validation could provide a more reliable estimate of general performance.

8. Conclusion

The project successfully demonstrated a basic supervised Machine Learning workflow for classifying Iris flowers. The dataset was explored and visualized, the features and target were prepared, and two classification

models were trained and evaluated. Decision Tree achieved 93.33% accuracy and KNN achieved 100.00% accuracy on the test data. KNN performed better in this experiment. The project provided practical experience with Python, data analysis, visualization, model training and model evaluation.

9. Future Scope

• Use cross-validation to evaluate model performance more reliably.

• Test additional classification algorithms and compare their results.

• Try different K values for KNN and tune model parameters.

• Build a simple user interface where users can enter flower measurements and receive a predicted species.

10. References

1. UCI Machine Learning Repository, Iris Dataset: https://archive.ics.uci.edu/dataset/53/iris

2. Scikit-learn documentation: https://scikit-learn.org/stable/

3. Pandas documentation: https://pandas.pydata.org/docs/

4. Matplotlib documentation: https://matplotlib.org/stable/

Repository: https://github.com/adhikarysayantani7-tony/aicte-internship-work

Author: Sayantani Adhikary